2022

SISTEMAS EN CHIP

PRACTICA 14 “VUMETRO”

FRANCISCO UZIEL CORDOVA PICHARDO

Objetivo

Al término de la sesión, los integrantes del equipo contaran con la habilidad de hacer uso del convertidor analógico digital del microcontrolador, implementándolo como un Vúmetro de dos canales.

Introducción Teórica

Un vúmetro es un dispositivo que se utiliza para medir y mostrar la amplitud de una señal de audio. Uno de los componentes principales de un vúmetro es el convertidor analógico-digital (ADC), que se encarga de convertir la señal analógica de audio en una señal digital para su procesamiento. El Atmega8535 es un microcontrolador de la familia AVR de Atmel que cuenta con un ADC incorporado, lo que lo hace adecuado para ser utilizado en proyectos de vúmetros.

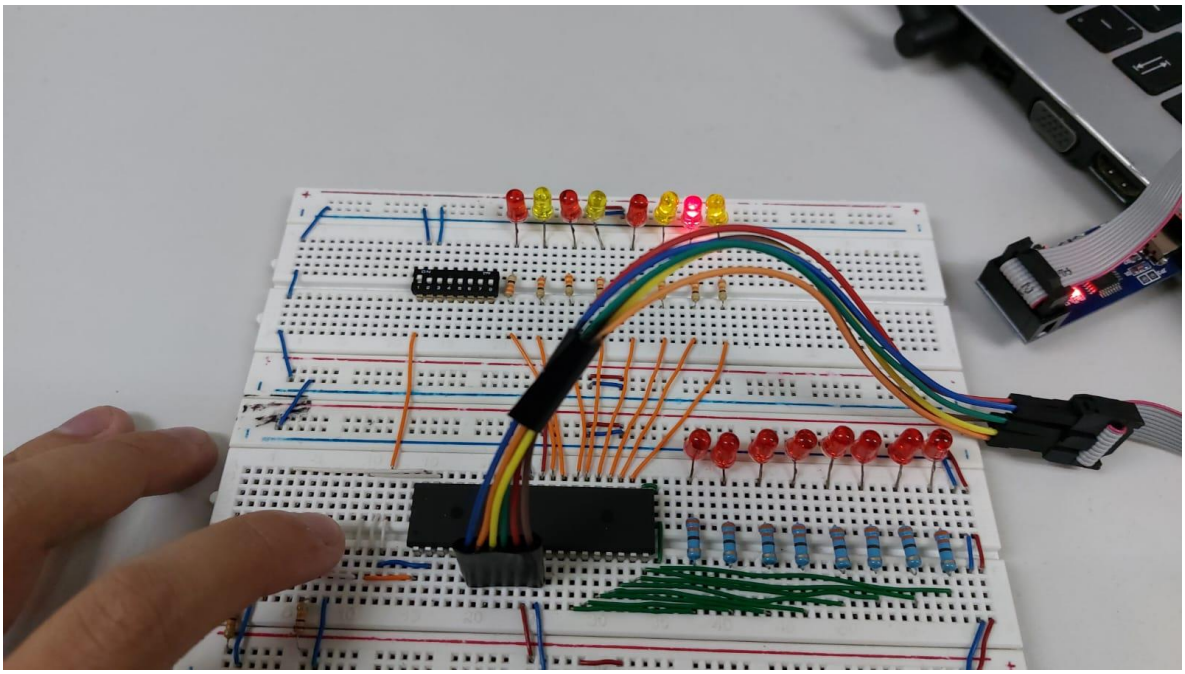
Para implementar un vúmetro con el Atmega8535, se utilizará el ADC para medir la señal de audio y convertirla en una señal digital. Luego, el microcontrolador podría procesar la señal digital para calcular la amplitud de la señal y controlar la salida de un display o leds para mostrar la lectura del vúmetro. El Atmega8535 también podría ser programado para agregar características como el promedio de la amplitud de la señal o la visualización de una barra de progreso.

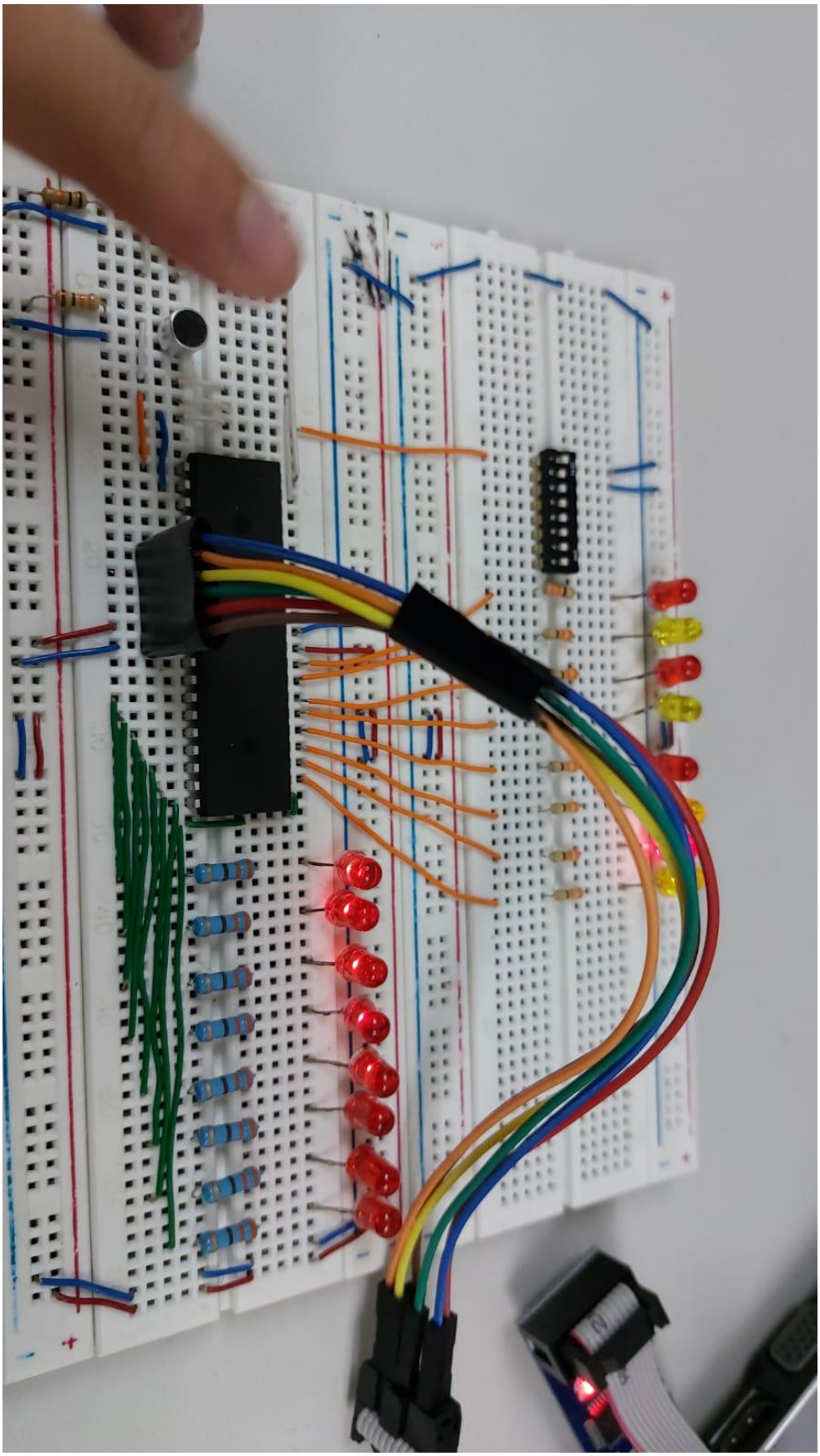
En resumen, el Atmega8535 con su ADC incorporado es una excelente opción para implementar un vúmetro ya que proporciona la precisión y velocidad necesaria para la medición y procesamiento de señales analógicas. Además, el microcontrolador ofrece una gran flexibilidad y posibilidades para agregar características adicionales al vúmetro.

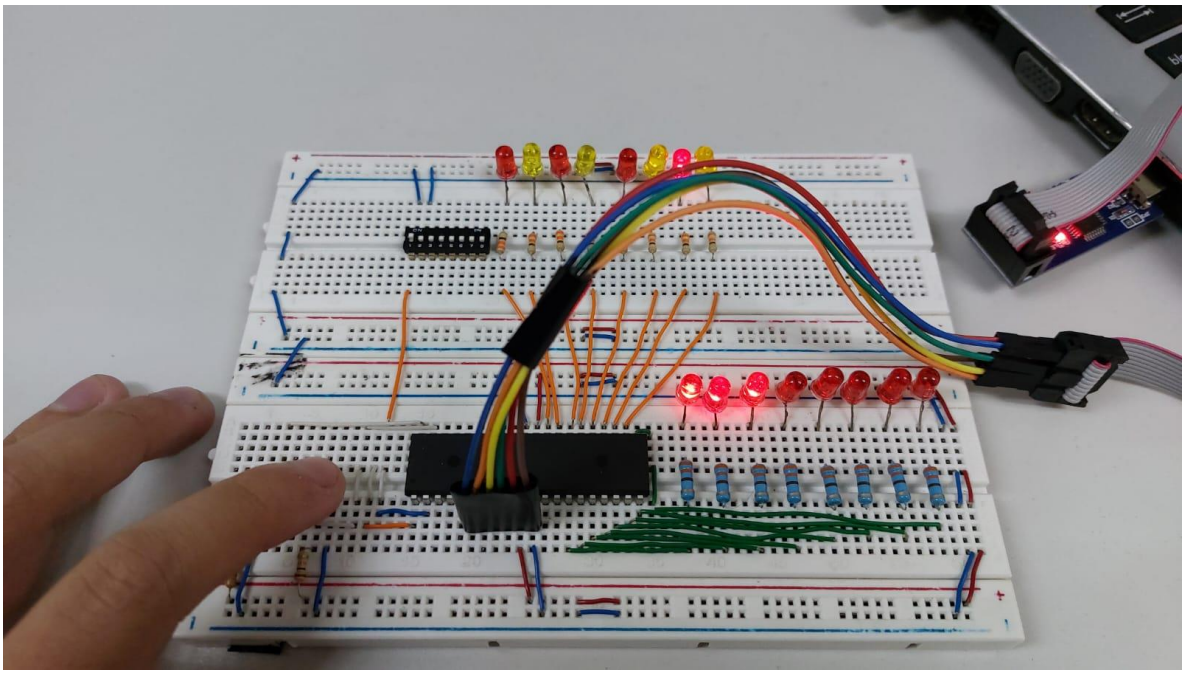
Desarrollo Experimental

1.- Realice una conversión de 8 bits sobre los canales 0 y 1 del ADC, establezca su voltaje de referencia para mostrar el resultado con leds en el Puerto B y D.

Fotografías del circuito funcionando







Código del programa

```
/*****************************************************
This program was created by the CodeWizardAVR V3.32
Automatic Program Generator
© Copyright 1998-2017 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project :
Version :
Date    : 08/12/2022
Author  :
Company :
Comments:


Chip type               : ATmega8535
Program type            : Application
AVR Core Clock frequency: 8.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 128
```

```
*****************************************************/

#include <mega8535.h>

#include <delay.h>
#define btn_tomar_a PINB.0
#define btn_hold PINB.1
#define btn_punto PINB.2
#define btn_barra PINB.3
char anterior_a, anterior_b, cantidad_a, cantidad_b;
bit saltar = 0;
// Declare your global variables here

// Voltage Reference: AVCC pin
#define ADC_VREF_TYPE ((0<<REFS1) | (1<<REFS0) | (1<<ADLAR))

// Read the 8 most significant bits
// of the AD conversion result
unsigned char read_adc(unsigned char adc_input)
{
ADMUX=adc_input | ADC_VREF_TYPE;
// Delay needed for the stabilization of the ADC input voltage
delay_us(10);
// Start the AD conversion
ADCSRA|=(1<<ADSC);
// Wait for the AD conversion to complete
while ((ADCSRA & (1<<ADIF))==0);
ADCSRA|=(1<<ADIF);
return ADCH;
}

unsigned char calcular_intensidad(unsigned char valor, unsigned char
cantidad, char modo) {
   char num_de_unos = valor / 31;
   unsigned char intensidad = 0;
   char i;

   num_de_unos = cantidad < num_de_unos ? num_de_unos - cantidad : 0;

   for (i = 0; i < num_de_unos; i++) {
    if (modo == 0) {
        if (i == 0)
            intensidad = (intensidad << 1) | 0x01;
        else
            intensidad = intensidad << 1;
```

```
    } else { //Barra
        intensidad = (intensidad << 1) | 0x01;
    }
   }

   return intensidad;
}

void main(void)
{
// Declare your local variables here

// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) |
(0<<DDA1) | (0<<DDA0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) |
(0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (0<<DDB3) | (0<<DDB2) |
(0<<DDB1) | (0<<DDB0);
// State: Bit7=P Bit6=P Bit5=P Bit4=P Bit3=P Bit2=P Bit1=P Bit0=P
PORTB=(1<<PORTB7) | (1<<PORTB6) | (1<<PORTB5) | (1<<PORTB4) | (1<<PORTB3) |
(1<<PORTB2) | (1<<PORTB1) | (1<<PORTB0);

// Port C initialization
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out
Bit0=Out
DDRC=(1<<DDC7) | (1<<DDC6) | (1<<DDC5) | (1<<DDC4) | (1<<DDC3) | (1<<DDC2) |
(1<<DDC1) | (1<<DDC0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) |
(0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out
Bit0=Out
DDRD=(1<<DDD7) | (1<<DDD6) | (1<<DDD5) | (1<<DDD4) | (1<<DDD3) | (1<<DDD2) |
(1<<DDD1) | (1<<DDD0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
```

```
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) |
(0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

// Timer/Counter 0 initialization
// Clock source: System Clock
// Clock value: Timer 0 Stopped
// Mode: Normal top=0xFF
// OC0 output: Disconnected
TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) |
(0<<CS01) | (0<<CS00);
TCNT0=0x00;
OCR0=0x00;

// Timer/Counter 1 initialization
// Clock source: System Clock
// Clock value: Timer1 Stopped
// Mode: Normal top=0xFFFF
// OC1A output: Disconnected
// OC1B output: Disconnected
// Noise Canceler: Off
// Input Capture on Falling Edge
// Timer1 Overflow Interrupt: Off
// Input Capture Interrupt: Off
// Compare A Match Interrupt: Off
// Compare B Match Interrupt: Off
TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) |
(0<<WGM10);
TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) |
(0<<CS11) | (0<<CS10);
TCNT1H=0x00;
TCNT1L=0x00;
ICR1H=0x00;
ICR1L=0x00;
OCR1AH=0x00;
OCR1AL=0x00;
OCR1BH=0x00;
OCR1BL=0x00;

// Timer/Counter 2 initialization
// Clock source: System Clock
// Clock value: Timer2 Stopped
// Mode: Normal top=0xFF
// OC2 output: Disconnected
ASSR=0<<AS2;
```

```
TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) |
(0<<CS21) | (0<<CS20);
TCNT2=0x00;
OCR2=0x00;

// Timer(s)/Counter(s) Interrupt(s) initialization
TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) |
(0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);

// External Interrupt(s) initialization
// INT0: Off
// INT1: Off
// INT2: Off
MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
MCUCSR=(0<<ISC2);

// USART initialization
// USART disabled
UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) |
(0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);

// Analog Comparator initialization
// Analog Comparator: Off
// The Analog Comparator's positive input is
// connected to the AIN0 pin
// The Analog Comparator's negative input is
// connected to the AIN1 pin
ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) |
(0<<ACIS1) | (0<<ACIS0);

// ADC initialization
// ADC Clock frequency: 1000.000 kHz
// ADC Voltage Reference: AVCC pin
// ADC High Speed Mode: Off
// ADC Auto Trigger Source: ADC Stopped
// Only the 8 most significant bits of
// the AD conversion result are used
ADMUX=ADC_VREF_TYPE;
ADCSRA=(1<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) |
(0<<ADPS2) | (1<<ADPS1) | (1<<ADPS0);
SFIOR=(1<<ADHSM) | (0<<ADTS2) | (0<<ADTS1) | (0<<ADTS0);

// SPI initialization
// SPI disabled
```

```
SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) |
(0<<SPR1) | (0<<SPR0);

// TWI initialization
// TWI disabled
TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

while (1)
      {
      unsigned char valor;
        unsigned char intensidad;
        unsigned char modo;

        modo = 0; //Barra
        if (!btn_punto) modo = 1;
        if (!btn_hold) modo = 2;

        if (saltar) {
            saltar = ~saltar;
            delay_ms(5);
            continue;
        }

            valor = read_adc(0);

            if (valor == anterior_a) {
                if (cantidad_a < 8) cantidad_a++;
            } else
                cantidad_a = 0;

            intensidad = calcular_intensidad(valor, cantidad_a, modo);
            PORTC = intensidad;
            anterior_a = valor;

            valor = read_adc(1);

            if (valor == anterior_b) {
                if (cantidad_b < 8) cantidad_b++;
            } else
                cantidad_b = 0;

            intensidad = calcular_intensidad(valor, cantidad_b, modo);
            PORTD = intensidad;
            anterior_b = valor;
```

```
        saltar = ~saltar;
        delay_ms(5);

      }
}
```

Conclusiones

En conclusión, el Atmega8535 es un excelente microcontrolador para implementar un vúmetro debido a su convertidor analógico-digital (ADC) incorporado. El ADC permite medir y procesar la señal de audio con precisión y velocidad, lo que es crucial para un vúmetro. Además, el microcontrolador es altamente programable y tiene una gran capacidad de procesamiento, lo que permite agregar características adicionales al vúmetro, como la visualización de una barra de progreso o el cálculo del promedio de la amplitud de la señal. En resumen, el Atmega8535 es una excelente opción para proyectos que requieren la medición y procesamiento de señales analógicas, como un vúmetro.

Bibliografía

Manual del ATmega8535: https://pdf1.alldatasheet.es/datasheet-pdf/view/164169/ATMEL/ATMEGA8535.html